

DERIVING INTERPRETERS: FROM BIG-STEP SOS TO ABSTRACT MACHINES

Master's Thesis

by

Mahmoud Mohamed Mohamed Elsayed Hamido

May 9, 2026

RPTU University Kaiserslautern-Landau
Department of Computer Science
67663 Kaiserslautern
Germany

Examiner: Prof. Dr. Ralf Hinze

Eigenständigkeitserklärung

Hiermit versichere ich, dass ich die von mir vorgelegte Arbeit mit dem Thema „Deriving Interpreters:

From Big-Step SOS to Abstract Machines“ selbstständig verfasst habe, dass ich die verwendeten Quellen und Hilfsmittel vollständig angegeben habe und dass ich die Stellen der Arbeit — einschließlich Tabellen und Abbildungen —, die anderen Werken oder dem Internet im Wortlaut oder dem Sinn nach entnommen sind unter Angabe der Quelle als Entlehnung kenntlich gemacht habe.

Kaiserslautern, den 9.5.2026

Mahmoud Mohamed Mohamed Elsayed Hamido

Abstract

Implementations of programming languages are expected to faithfully realize their specifications, yet in practice this correspondence is often validated only by testing against written standards or reference implementations. Since testing ranges over finitely many programs, it cannot establish the absence of mismatches for all well-typed inputs. This thesis develops a machine-checked approach in Agda in which an implementation is systematically derived from its formal semantics and proved correct by construction. We present a uniform derivation of abstract machines from semantic definitions, carried out for three case studies: arithmetic expressions, arithmetic expressions with exceptions, and the simply typed lambda calculus. The derivation makes evaluation structure explicit via typed frames and a stack of machine states, with operational modes for evaluation and return, and an additional unwind mode for exceptions. For each language, transition rules are obtained by mimicking in-order evaluation, and the resulting machine is proved correct and complete with respect to big-step semantics. As a corollary, we establish semantic equivalences between big-step, small-step, denotational, and abstract-machine presentations, showing that the semantics are compositional and mutually consistent. These results provide a systematic methodology for deriving verified interpreters from formal specifications.

Zusammenfassung

Von Implementierungen von Programmiersprachen wird erwartet, dass sie ihre Spezifikationen treu umsetzen; in der Praxis wird diese Übereinstimmung jedoch oft nur durch Tests anhand von Spezifikationsdokumenten oder Referenzimplementierungen überprüft. Da sich die Tests auf eine endliche Anzahl von Programmen beschränken, können sie nicht für alle wohlgetypten Eingaben das Fehlen von Abweichungen nachweisen. Diese Arbeit entwickelt einen maschinell überprüfbaren Ansatz in Agda, bei dem eine Implementierung systematisch aus ihrer formalen Semantik abgeleitet und durch Konstruktion als korrekt bewiesen wird. Wir präsentieren eine einheitliche Ableitung abstrakter Maschinen aus semantischen Definitionen, die für drei Fallstudien durchgeführt wurde: arithmetische Ausdrücke, arithmetische Ausdrücke mit Ausnahmen und der einfach typisierte Lambda-Kalkül. Die Ableitung macht die Auswertungsstruktur durch typisierte Frames und einen Stapel von Maschinenzuständen explizit, mit Betriebsmodi für die Auswertung und die Rückgabe sowie einem zusätzlichen Unwind-Modus für Ausnahmen. Für jede Sprache werden Übergangsregeln durch Nachahmung der In-Order-Auswertung gewonnen, und die resultierende Maschine wird in Bezug auf die Big-Step-Semantik als korrekt und vollständig bewiesen. Als Korollar stellen wir semantische Äquivalenzen zwischen Big-Step-, Small-Step-, denotationalen und abstrakten Maschinen-Darstellungen her und zeigen, dass die Semantiken kompositorisch und wechselseitig konsistent sind. Diese Ergebnisse liefern eine systematische Methodik zur Ableitung verifizierter Interpreter aus formalen Spezifikationen.

Contents

1	Introduction	1
2	Background	3
2.1	Formalizing Language Semantics	6
2.2	Deriving Abstract Machines	8
3	Arithmetic	11
3.1	Syntax and Types	11
3.2	Big-Step Semantics	11
3.3	Small-Step Semantics	12
3.4	Denotational Semantics	15
3.5	Abstract Machine	16
4	Exceptions	19
4.1	Syntax	19
4.2	Big-Step Semantics	19
4.3	Small-Step Semantics	20
4.4	Denotational Semantics	22
4.5	Abstract Machine	23
5	The Simply-Typed Lambda Calculus	27
5.1	Syntax and Types	27
5.2	Big-Step Semantics	27
5.3	Small-Step Semantics	28
5.4	Denotational Semantics	30
5.5	Abstract Machine	32
6	Related Work	35
7	Conclusion	37
	Bibliography	39

1 Introduction

Ideally, an implementation of a programming language faithfully realizes its specification. A mismatch between the two — a program whose behaviour according to the specification differs from what a compiler or interpreter produces — is in most cases a bug, leading programs to behave unpredictably across different environments, such as different operating systems or hardware architectures, or between different implementations. But how would one check that no such mismatch exists?

In practice, specifications are written documents: the C++ standard, the Java Language Specification, and the WebAssembly specification all describe language behaviour in plain text, occasionally supplemented by evaluation rules. An implementation is built to match that description, and conformance is gauged through testing. A more systematic approach, such as the one used for WebAssembly [Ros22], is to provide a *reference interpreter* alongside the specification: a new implementation is deemed compliant if it produces the same output as the reference on a shared test suite. Both approaches share the following flaw. Testing cannot prove the *absence* of a mismatch between the specification and the implementation. A test suite exercises a finite set of inputs, while a specification makes claims about all possible programs; no matter how thorough the tests, some combination of language features may expose a discrepancy that no test has yet reached.

Formal verification addresses this by allowing it to be *proven* that an implementation (by construction) conforms to the specification, covering all inputs and all cases simultaneously. In a *dependently typed* language such as Agda [Stu16], types can encode propositions on values. A specification can therefore be expressed as a type (or more specifically, a relation), and a compliant implementation is one that can be used to construct a value inhabiting that type. The theoretical foundation for this is the *Curry–Howard correspondence* [Wad15]: every logical proposition corresponds to a type, and a proof of that proposition corresponds to a value of that type.

Formal programming language semantics are the basis of such proofs, originally described using pen and paper, but can be encoded in a proof assistant. Several forms of semantics are in common use [Pie02]. *Natural* (big-step) semantics relates expressions directly to the values they evaluate to, and is generally the most concise style since only “productive” rules need to be stated. *Operational* (small-step) semantics describes computation as a sequence of individual reduction steps; it is more verbose, requiring both context and reduction rules, but exposes fine-grained control over evaluation order and intermediate states. *Denotational* semantics maps terms to a domain capable of expressing their “meaning” (e.g. functions, natural numbers), and is often

realised as a functional interpreter. *Abstract Machines* shows how a theoretical machine with a state and transitions between states can implement the semantics of a language. Assuming that the semantics for a language are consistent, they can be related, and in some cases one can be *derived* from another.

The central thesis of this work is that an implementation can be derived directly from a specification. We have chosen to focus on the derivation of a correct abstract machine from a big-step semantics. We develop this strategy through three case studies on the following languages: arithmetic expressions, arithmetic with exceptions, and the simply typed lambda calculus (STLC).

2 Background

Agda [Stu16] is a dependently typed programming language and proof assistant. Readers familiar with Haskell will find Agda’s syntax immediately recognisable. A notable surface difference is that Agda makes heavy use of Unicode and supports *mixfix* operators, where underscores mark the argument positions. For example, `if_then_else_` is a valid identifier that can be applied as `if b then x else y`.

The fundamental difference, however, lies in the type system. Agda types may *depend* on values: a function’s return type can mention its argument, and a data type can be indexed by a value computed at runtime. In Agda, `Set` designates the type of types.¹

The theoretical basis for using Agda as a proof assistant is the *Curry–Howard correspondence* [Wad15], also known as “Propositions as Types”: every logical proposition corresponds to a type, and a proof of that proposition is a value of that type. Agda’s logic is *constructive* (intuitionistic), meaning a proposition is true precisely when one can explicitly construct a witness (value) for it. The type checker then also serves as a “proof checker”. That is, a well-typed program is a valid proof.

The simplest proposition is truth, $\top$, represented by a type with exactly one constructor:

```
data  $\top$  : Set where
  tt :  $\top$ 
```

Because it is always possible to produce `tt`, $\top$ is trivially provable.

Falsehood, $\perp$, is the opposite, a type with *no* constructors at all:

```
data  $\perp$  : Set where
```

There is no way to produce a value of type $\perp$, so it cannot be proven. If a value of type $\perp$ were somehow obtained, any proposition would follow, which is the principle of *ex falso quodlibet*:

```
ex-falso : {P : Set}  $\rightarrow$   $\perp$   $\rightarrow$  P
ex-falso ()
```

The empty pattern `()` signals to Agda that there is no constructor to match.

Implication ($P \Rightarrow Q$) corresponds to the function type $P \rightarrow Q$: given a proof of P , produce a proof of Q . Negation $\neg P$ is implication into falsehood:

```
 $\neg$ _ : Set  $\rightarrow$  Set
 $\neg$  P = P  $\rightarrow$   $\perp$ 
```

¹To avoid a paradox arising from `Set : Set`, types form a cumulative hierarchy: `Set : Set 1`, `Set 1 : Set 2`, and so on.

Conjunction $P \wedge Q$ corresponds to a product type. A proof of $P \wedge Q$ is a pair of proofs.

```
data _×_ (P Q : Set) : Set where
  _,_ : P → Q → P × Q
```

The dependent generalisation of this is the Σ -type, where the type of the second component may depend on the value of the first:

```
data  $\Sigma$  (A : Set) (B : A → Set) : Set where
  _,_ : (x : A) → B x →  $\Sigma$  A B
```

```
 $\exists$  : {A : Set} → (A → Set) → Set
 $\exists$  {A = A} P =  $\Sigma$  A P
```

This corresponds to existential quantification. A proof of $\exists x. P(x)$ is a pair of a concrete witness x and a predicate P that holds of it. Disjunction $P \vee Q$ is a disjoint union.

```
data _ $\sqcup$ _ (P Q : Set) : Set where
  inl : P → P  $\sqcup$  Q
  inr : Q → P  $\sqcup$  Q
```

In contrast to classical logic, constructive logic does not admit the law of the excluded middle ($P \vee \neg P$), nor the double negation elimination ($\neg\neg P \Rightarrow P$).

A key distinction in Agda data declarations is between *parameters* and *indices*. Both may appear in the signature of a **data** declaration. Parameters are listed before the colon and are fixed across all constructors. Indices are listed after the colon and different constructors may specialise them to different values. The canonical example is **Vec**, which is a list type whose length is tracked in its type:

```
data Vec (A : Set) :  $\mathbb{N}$  → Set where
  [] : Vec A zero
  _::_ : A → Vec A n → Vec A (suc n)
```

Here **A** is a (type) parameter, all constructors of **Vec A n** store elements of the same type **A**. The natural number after the colon is an index: the constructor for the empty list `[]` fixes it to **zero**, while `_::_` advances it from **n** to **suc n**, so the length is tracked at the type level. This ability to encode invariants into indices is used extensively throughout this thesis.

Propositional equality is definable as an indexed type. The `_≡_` relation is indexed by two terms of the same type, with a single constructor **refl** specialising both indices to the same value, witnessing that a term is equal to itself:

```
data _≡_ {A : Set} (x : A) : A → Set where
  refl : x ≡ x
```

To make this concrete, consider a proof that addition of natural numbers is commutative. The definitions nearly mirror their Haskell counterparts.

```

data N : Set where
  zero : N
  suc  : N → N

_+_ : N → N → N
zero + m = m
suc n + m = suc (n + m)

```

Addition is defined by recursion on the first argument.

Commutativity does not follow immediately from the definition, so we first prove two helper lemmas by structural induction. The `rewrite` keyword takes an equality proof of the form $t \equiv u$ and performs a substitution in the current goal.

```

n+0≡n : (n : N) → n + zero ≡ n
n+0≡n zero = refl
n+0≡n (suc n) rewrite n+0≡n n = refl

n+sm≡n : (n m : N) → n + suc m ≡ suc (n + m)
n+sm≡n zero m = refl
n+sm≡n (suc n) m rewrite n+sm≡n n m = refl

```

In each inductive case, rewriting with the induction hypothesis reduces the goal to `refl`. The main theorem then follows:

```

+-commutative : (n m : N) → n + m ≡ m + n
+-commutative zero m rewrite n+0≡n m = refl
+-commutative (suc n) m rewrite +-commutative n m = sym (n+sm≡n m n)

```

One additional obligation Agda imposes beyond type correctness is *termination*. Paired with `ex-falso`, a non-terminating function could be used to prove any proposition. Consider the following definition of a diverging function:

```

diverging : ⊥
diverging = diverging

```

To prevent this, Agda's termination checker requires that every recursive call is made on a structurally smaller argument. The termination checker may reject a definition that is actually terminating; in such cases a `{-# TERMINATING #-}` pragma can be used to disable the check, leaving the burden of proof to the programmer.

2.1 Formalizing Language Semantics

We use a variant of the Simply Typed Lambda Calculus (STLC) as our running example. The types of the language are natural numbers, booleans, and function types:

$$T ::= \text{Nat} \mid \text{Bool} \mid T_1 \Rightarrow T_2$$

The concrete representation of these definitions in Agda will appear in later chapters, namely, Chapter 3, Chapter 4, and Chapter 5.

De Bruijn Indices

Named variables require care around α -equivalence and capture-avoiding substitution. De Bruijn indices [Bru75] address these issues by replacing variable names with indices. Each index represents how many λ -binders lie between the variable and the λ that introduced it. For example, the identity function $\lambda x. x$ is written $\lambda 0$, and the constant function $\lambda x. \lambda y. x$ is written $\lambda \lambda 1$.

Contexts record the types of bound variables:

$$\Gamma ::= \emptyset \mid \Gamma, T$$

A well-typed term is expressed as a typing judgment $\Gamma \vdash T$, where Γ lists the types of its free variables. The syntax is:

$$e ::= n \mid b \mid e_1 \oplus e_2 \mid \text{if } e \text{ then } e_1 \text{ else } e_2 \mid \text{Var } x \mid \lambda e \mid e_1 \cdot e_2$$

where n denotes a natural number, b a boolean literal, λe a lambda abstraction (with the bound variable as De Bruijn index 0 inside e), and $e_1 \cdot e_2$ is function application.

Big-Step Semantics

Natural (big-step) semantics is a relation on a term and the value it evaluates to, with subderivations recording the evaluation of each subterm. Non-terminating computations are not representable within big-step semantics.

We assign a value domain $\models : \text{Type} \rightarrow \text{Set}$ to each type:

$$\models T = \begin{cases} \mathbb{N} & T = \text{Nat} \\ \mathbb{B} & T = \text{Bool} \\ \text{Closure}(T_1, T_2) & T = T_1 \Rightarrow T_2 \end{cases}$$

where a closure $\langle \delta, e \rangle$ pairs a saved environment δ with a term e . An environment $\delta : \Gamma$ maps each variable $x : T \in \Gamma$ to a value $v : \models T$.

The evaluation judgment $\delta \vdash e \Downarrow v$ reads: under environment δ , expression e evaluates to value v .

$$\begin{array}{c}
 \frac{}{\delta \vdash n \Downarrow n} \text{NAT} \qquad \frac{}{\delta \vdash b \Downarrow b} \text{BOOL} \qquad \frac{}{\delta \vdash \text{Var } x \Downarrow \delta(x)} \text{VAR} \\
 \\
 \frac{}{\delta \vdash \lambda e \Downarrow \langle \delta, e \rangle} \text{ABS} \\
 \\
 \frac{\delta \vdash e_1 \Downarrow \langle \delta', e' \rangle \quad \delta \vdash e_2 \Downarrow v_2 \quad (\delta', v_2) \vdash e' \Downarrow v}{\delta \vdash e_1 \cdot e_2 \Downarrow v} \text{APP} \\
 \\
 \frac{\delta \vdash e_1 \Downarrow n_1 \quad \delta \vdash e_2 \Downarrow n_2}{\delta \vdash e_1 \oplus e_2 \Downarrow n_1 + n_2} \text{PLUS} \qquad \frac{\delta \vdash e \Downarrow \text{true} \quad \delta \vdash e_1 \Downarrow v}{\delta \vdash \text{if } e \text{ then } e_1 \text{ else } e_2 \Downarrow v} \text{IFTRUE} \\
 \\
 \frac{\delta \vdash e \Downarrow \text{false} \quad \delta \vdash e_2 \Downarrow v}{\delta \vdash \text{if } e \text{ then } e_1 \text{ else } e_2 \Downarrow v} \text{IFFALSE}
 \end{array}$$

Small-Step Semantics

Structural operational (small-step) semantics describes computation as a sequence of individual reduction steps. Unlike big-step semantics, it can capture fine-grained evaluation order and makes intermediate states explicit, but is more verbose: rules split into β -rules, which perform reductions, and ξ -rules (congruence rules), which specify a term reduction in some subterm.

Values are now a *syntactic* subset of expressions. n , b , and λe are the values (normal forms). The single-step reduction judgment $e \longrightarrow e'$ is defined by:

$$\begin{array}{c}
 \frac{v_2 : \models T_1}{(\lambda e_1) \cdot v_2 \longrightarrow e_1[v_2/0]} \beta\text{-APP} \qquad \frac{}{n_1 \oplus n_2 \longrightarrow n_1 + n_2} \beta\text{-PLUS} \\
 \\
 \frac{}{\text{if true then } e_1 \text{ else } e_2 \longrightarrow e_1} \beta\text{-IFTRUE} \\
 \\
 \frac{}{\text{if false then } e_1 \text{ else } e_2 \longrightarrow e_2} \beta\text{-IFFALSE} \qquad \frac{e_1 \longrightarrow e'_1}{e_1 \cdot e_2 \longrightarrow e'_1 \cdot e_2} \xi\text{-APP}_1 \\
 \\
 \frac{v_1 : \models (T_1 \Rightarrow T_2) \quad e_2 \longrightarrow e'_2}{v_1 \cdot e_2 \longrightarrow v_1 \cdot e'_2} \xi\text{-APP}_2 \qquad \frac{e_1 \longrightarrow e'_1}{e_1 \oplus e_2 \longrightarrow e'_1 \oplus e_2} \xi\text{-PLUS}_1 \\
 \\
 \frac{e_2 \longrightarrow e'_2}{n_1 \oplus e_2 \longrightarrow n_1 \oplus e'_2} \xi\text{-PLUS}_2 \\
 \\
 \frac{e \longrightarrow e'}{\text{if } e \text{ then } e_1 \text{ else } e_2 \longrightarrow \text{if } e' \text{ then } e_1 \text{ else } e_2} \xi\text{-IF}
 \end{array}$$

Note that unlike big-step semantics which constructs closure values, and looking up variables in the environment, the β -rule for application performs substitution directly on the body of the function. $e_1[v_2/0]$ denotes substituting v_2 for

De Bruijn index 0 in e_1 , with the remaining indices shifted accordingly. The reflexive-transitive closure $\longrightarrow^*$ then captures complete evaluation sequences as a series of individual reduction steps.

Denotational Semantics

Denotational semantics maps terms to a domain capable of expressing their meaning, and is often realised as an interpreter in a meta-language. Choosing the appropriate domain is not always straightforward.

For a language with basic types, the value domain for each type is a corresponding Agda type, and the interpreter is a structurally recursive function over the expression tree. For languages such as STLC, there are two viable representations for function types. The natural choice is to interpret function types as Agda function types:

$$\llbracket A \Rightarrow B \rrbracket = \llbracket A \rrbracket \rightarrow \llbracket B \rrbracket$$

Alternatively, function types can be represented as explicit closures $\langle \delta, e \rangle$, pairing the captured environment with a term to be evaluated when the function is applied.

$$\llbracket A \Rightarrow B \rrbracket = \text{Closure } A B$$

The interpreter takes an environment δ assigning a value of type $\llbracket T \rrbracket$ to each variable of type T in Γ , and returns a value. A lambda abstraction maps to an Agda function that extends the environment and calls the interpreter on the body; application evaluates the function position to a closure $\langle \delta', e' \rangle$, evaluates the argument to v , and then calls the interpreter on the stored body e' under the extended environment (δ', v) .

2.2 Deriving Abstract Machines

Consider an interpreter as a recursive function that traverses the expression tree: evaluating a compound expression requires evaluating each subexpression in some defined order (typically left-to-right). For example, in an addition expression, the left operand is evaluated before the right, and the right before the sum is returned to the caller. Each recursive call to the interpreter suspends computation at the call site until the sub-result is available, after which the rest of the subexpressions can be evaluated.

These suspended computations can be made explicit as *frames*. A frame records the surrounding context—what computation remains once a particular subexpression has yielded a value—while leaving a hole for that value.

For addition, entering the left operand pushes a frame of the form $(\circ + e_2)$; once the left value v_1 is known, that frame is replaced by $(v_1 + \circ)$, which awaits the right operand. The collection of outstanding frames forms a stack.

The interpreter can now be structured as an abstract machine. The state of the machine is a stack of frames, and the current expression being evaluated, or a value being returned. Transition rules decompose the expression, push a

frame, and select a subexpression to evaluate next. A transition from a return state pops the top frame, fills its hole with the returned value, and either produces a new return state or an eval state for the remaining subexpression. The machine halts when it reaches a return state with an empty stack.

Once a machine is defined, two properties must be proven for the machine to be consistent with the big-step semantics:

- *Completeness*: if $e \Downarrow v$, then for any stack stk , the machine started evaluating e with stack stk reaches the return state $stk \downarrow v$.
- *Correctness*: if the machine started evaluating e with an *empty* stack, it reaches a final return state carrying v , then $e \Downarrow v$.

Proof Strategy: A Chain of Equivalences

Rather than proving each pair of semantics equivalent in isolation, we also use a *chain of equivalences* to relate most (if not all) forms of semantics.

$$\text{Denotational} \longleftrightarrow \text{Big-Step} \longleftrightarrow \text{Machine} \longleftrightarrow \text{Small-Step}$$

Each link in the chain is proved independently. Transitivity then yields equivalence between any two points. In the event that a direct proof is difficult, it can be obtained by composing the relevant links. This is similar to how *intermediate representations* in a compiler perform incremental transformations — rather than translating directly from source to assembly in a single step.

3 Arithmetic

3.1 Syntax and Types

We define a simple arithmetic language with natural numbers and booleans, addition, and conditionals:

```
data Type : Set where
  Nat : Type
  Bool : Type

⊢_ : Type → Set
⊢ Nat = ℕ
⊢ Bool = B

data ⊢_ : Type → Set where
  \_      : ⊢ T → ⊢ T
  _⊞_     : ⊢ Nat → ⊢ Nat → ⊢ Nat
  test    : ⊢ Nat → ⊢ Bool
  if_then_else_ : ⊢ Bool → ⊢ T → ⊢ T → ⊢ T

test' : ℕ → B
test' zero = true
test' (suc n) = false
```

The expression datatype $\vdash_$ is *intrinsically typed*: it is *indexed* by a **Type**, where each constructor specifies the type of the expression it produces. A literal “quote” $\backslash_$ embeds a value of type T directly into the syntax at matching index T . As a consequence, every term that can be constructed must be well typed; ill-typed programs are not representable. Note that in the latter two constructors, the types enforce certain constraints. For example, that the condition of an **if**-expression must be an expression with a boolean type, and that both branches must have the same type.

3.2 Big-Step Semantics

The big-step semantics is modelled as an inductive relation between expressions and their evaluated values. Each rule corresponds to a constructor of the relation.

```
data _⇓_ : ⊢ T → ⊢ T → Set where
  \_ : (v : ⊢ T) → (\ v) ⇓ v
  _⊞_ : e₁ ⇓ n₁ → e₂ ⇓ n₂
       → (e₁ ⊞ e₂) ⇓ (n₁ + n₂)
  test : e ⇓ n
```

$$\begin{aligned}
& \rightarrow (\text{test } e) \Downarrow (\text{test}' n) \\
\text{if-then} & : e \Downarrow \text{true} \rightarrow e_1 \Downarrow v \\
& \rightarrow (\text{if } e \text{ then } e_1 \text{ else } e_2) \Downarrow v \\
\text{if-else} & : e \Downarrow \text{false} \rightarrow e_2 \Downarrow v \\
& \rightarrow (\text{if } e \text{ then } e_1 \text{ else } e_2) \Downarrow v
\end{aligned}$$

A value which is either a natural number or a boolean is a normal form, and evaluates to itself. The operands of addition are evaluated recursively, and the results are added. The if-then-else expression evaluates the condition first, and then evaluates either the "then" or "else" branch based on the result of the condition.

Evaluation is *deterministic*: if an expression evaluates to two different values, they must be equal.

$$\begin{aligned}
\text{deterministic} & : \forall \{e : \vdash T\} \{v_1 \ v_2 : \models T\} \\
& \rightarrow e \Downarrow v_1 \rightarrow e \Downarrow v_2 \rightarrow v_1 \equiv v_2
\end{aligned}$$

This is proven by structural induction on the evaluation relation. In the case of values, both values must be the same by injectivity of the constructor. In the case of addition, we perform induction on the operands, to show that the same operands must evaluate to the same values, and therefore the same sum. In the case of conditionals, we show that the condition must evaluate to the same boolean. If that is not the case, we would have an assumption $\text{false} \equiv \text{true}$ which can be discharged by pattern matching on it. Finally, we show that the same branch must be taken in both evaluations, and that the same value is returned from that branch.

Every expression evaluates to *some* value.

$$\text{total} : \forall (e : \vdash T) \rightarrow \exists (\lambda v \rightarrow e \Downarrow v)$$

The proof for total is similar; but we perform structural induction on the expression instead, and produce the value and the derivation tree from recursive calls onto the subexpressions.

3.3 Small-Step Semantics

The reduction relation $e \longrightarrow e'$ has the following rules:

$$\begin{array}{c}
 \frac{}{('n_1) ('n_2) \longrightarrow ('(n_1 + n_2))} \beta\text{-}\boxplus \qquad \frac{}{\text{test}('0) \longrightarrow 'true} \beta\text{-test}_0 \\
 \frac{}{\text{test}('(\text{suc } n)) \longrightarrow 'false} \beta\text{-test}_+ \\
 \frac{}{\text{if}('true) \text{ then } e_1 \text{ else } e_2 \longrightarrow e_1} \beta\text{-if-then} \\
 \frac{}{\text{if}('false) \text{ then } e_1 \text{ else } e_2 \longrightarrow e_2} \beta\text{-if-else} \qquad \frac{e_1 \longrightarrow e'_1}{e_1 \text{ } e_2 \longrightarrow e'_1 \text{ } e_2} \xi\text{-}\boxplus_1 \\
 \frac{e_2 \longrightarrow e'_2}{('n) \text{ } e_2 \longrightarrow ('n) \text{ } e'_2} \xi\text{-}\boxplus_2 \qquad \frac{e \longrightarrow e'}{\text{test } e \longrightarrow \text{test } e'} \xi\text{-test} \\
 \frac{e \longrightarrow e'}{\text{if } e \text{ then } e_1 \text{ else } e_2 \longrightarrow \text{if } e' \text{ then } e_1 \text{ else } e_2} \xi\text{-if}
 \end{array}$$

Complete evaluation is captured by the reflexive-transitive closure $\longrightarrow^*$, defined by a **base** case (zero steps) and a **step** constructor that prepends one reduction. The equivalence proof between big-step and small-step semantics is defined as so:

```

simulate : ∀ {T : Type} {e : ⊢ T} {v : ⊢ T} → e ↓ v → e →* v
simulate (' v) = base
simulate (p₁ ⊕ p₂) =
  →*-transitive (→*-⊕₁ (simulate p₁))
  (→*-transitive (→*-⊕₂ (simulate p₂))
  (step β-⊕ base))
simulate (test {n = zero} p) =
  →*-transitive (→*-test (simulate p))
  (step β-test-zero base)
simulate (test {n = suc _} p) =
  →*-transitive (→*-test (simulate p))
  (step β-test-suc base)
simulate (if-then pe p₁) =
  →*-transitive (→*-if (simulate pe))
  (step β-if-then (simulate p₁))
simulate (if-else pe p₂) =
  →*-transitive (→*-if (simulate pe))
  (step β-if-else (simulate p₂))
    
```

Note the use of the ξ -rules lifted to the reflexive-transitive closure $\longrightarrow^*$. Each case effectively applies the relevant ξ -rule in the expected evaluation order. The definition of these lifted ξ -rules is straightforward: they are defined by induction on the relation, with a base case involving no further steps, and in the inductive case, we apply the unlifted ξ -rule to the first reduction step, and then recursively perform the lifted ξ -rule on the remaining steps.

```

→*-⊕₁ : ∀ {e₁ e₁' e₂ : ⊢ Nat} → e₁ →* e₁' → (e₁ ⊕ e₂) →* (e₁' ⊕ e₂)
    
```

```

→*⊞1 base = base
→*⊞1 (step r p) = step (ξ⊞1 r) (→*⊞1 p)

→*⊞2 : ∀ {n : ℕ} {e2 e2' : ⊢ Nat} → e2 →* e2' → (λ n ⊞ e2) →* (λ n ⊞ e2')
→*⊞2 base = base
→*⊞2 (step r p) = step (ξ⊞2 r) (→*⊞2 p)
    
```

The other direction of the equivalence is defined by induction on the small-step relation, with a case for each β -rule, and a case for each ξ -rule that applies the induction hypothesis to the first reduction step, and then uses transitivity to concatenate the resulting big-step derivation with the one corresponding to the β -rule.

```

simulateR : ∀ {T : Type} {e : ⊢ T} {v : ⊢ T} → e →* λ v → e ⇓ v
    
```

The proof relies on the following lemmas:

- The big-step relation is **total**. There always exists some value v that an expression evaluates to, and the derivation tree for that evaluation from e to that v .
- The small-step relation is **deterministic**. If, in a single step there are two derivations from the same expression $e \rightarrow^* e_1$ and $e \rightarrow^* e_2$, then e_1 and e_2 must be the same expression. This is trivial by pattern matching on both derivations.
- The transitive closure of the step relation is **confluent**, that is, for two derivations $e \rightarrow^* e_1$ and $e \rightarrow^* e_2$, and assuming e_1 and e_2 are values (i.e., there is no expression e' such that either e_1 or e_2 can step to e'): then e_1 and e_2 must be equal.

```

→*confluence : ∀ {T : Type} {e e1 e2 : ⊢ T}
→ e →* e1 → e →* e2
→ (∀ {e' : ⊢ T} → e1 → e' → ⊥)
→ (∀ {e' : ⊢ T} → e2 → e' → ⊥)
→ e1 ≡ e2

→*confluence base base _ _ = reflexive
→*confluence base (step r _) ns1 _ = ex-falso (ns1 r)
→*confluence (step r _) base _ ns2 = ex-falso (ns2 r)
→*confluence (step r1 p1) (step r2 p2) ns1 ns2 rewrite det r1 r2 =
→*confluence p1 p2 ns1 ns2
    
```

The proof for this lemma is by induction on both derivations. In the event that either of the derivations is a base case, but the other is not, then we have contradicted the assumption that the resulting expression is a value, and we can discharge the case by ex-falso. In the case that both derivations are a step, we can use the determinism of the single-step relation to show that the first reduction step must be the same in both derivations, followed by the induction hypothesis on the remaining steps.

Finally, we can prove `simulateR`.

```

simulateR : ∀ {T : Type} {e : ⊢ T} {v : ⊨ T} → e →* ` v → e ⇓ v
simulateR {e = e} {v = v} tr =
  let (v' , p) = total e

  tr' : e →* (` v')
  tr' = simulate p

  eq : (` v') ≡ (` v)
  eq = →*-confluence tr' tr →step-` →step-`
in subst (e ⇓_) (`-inj eq) p

```

Given that the big-step semantics is total, we can obtain the derivation tree for e which results in a value v' , as well as a small-step sequence of reduction steps from $e \longrightarrow^* v'$ by simulating the big-step derivation. The goal is now to show that the value produced by the big-step v' is the same as the value v produced by the small-step semantics. We can apply the confluence lemma to deduce that v' and v are the same value, since both are values reachable from e . The latter is realized by the following helper lemma.

```

→step-` : ∀ {T : Type} {v : ⊨ T} {e' : ⊢ T} → (` v) → e' → ⊥
→step-` ()

```

Since there is no constructor in relation that further reduces a value, the case is discharged by Agda.

3.4 Denotational Semantics

The denotational approach interprets expressions directly as values.

```

[ ] : ∀ {T} → ⊢ T → ⊨ T
[ ` v ] = v
[ e1 ⊞ e2 ] = [ e1 ] + [ e2 ]
[ test e ] = test' [ e ]
[ if e then e1 else e2 ] with [ e ]
... | true = [ e1 ]
... | false = [ e2 ]

```

The equivalence theorems between big-step and the denotational semantics state that if $e \Downarrow v$, then $\llbracket e \rrbracket \equiv v$, and vice versa.

```

simulate : ∀ {T : Type} {e : ⊢ T} {v : ⊨ T} → e ⇓ v → [ e ] ≡ v

```

The first direction of the proof is simple: we perform induction on the derivation tree and show that the interpreter produces the same value by reflexivity. For compound expressions, we rewrite using the induction hypothesis on the subexpressions.

```

simulate (` v) = reflexive
simulate (e1 ⊞ e2) rewrite simulate e1 | simulate e2 = reflexive
simulate (test e) rewrite simulate e = reflexive
simulate (if-then e1 e2) rewrite simulate e1 | simulate e2 = reflexive
simulate (if-else e1 e2) rewrite simulate e1 | simulate e2 = reflexive

```

The other direction of the proof is similar, but we construct the derivation tree using the corresponding constructor for each case.

```

simulateR : ∀ {T : Type} (e : ⊢ T) {v : ⊢ T} → [ e ] ≡ v → e ⇓ v

simulateR (⊢ x) reflexive = ⊢ x
simulateR (e1 ⊞ e2) p with [ e1 ] in eq1 | [ e2 ] in eq2 | p
... | v1 | v2 | reflexive = simulateR e1 eq1 ⊞ simulateR e2 eq2

simulateR (test e) p with [ e ] in eq | p
... | v | reflexive = test (simulateR e eq)

simulateR (if e then e1 else e2) p with [ e ] in eq | p
... | true | reflexive = if-then (simulateR e eq) (simulateR e1 reflexive)
... | false | reflexive = if-else (simulateR e eq) (simulateR e2 reflexive)

```

3.5 Abstract Machine

The following definitions show how to derive an abstract machine from the big-step semantics using the approach briefly described in the previous chapter.

```

data Hole : Set where
  ○ : Hole

data Frame : Type → Type → Set where
  _else_      : ⊢ T → ⊢ T → Frame T Bool
  _⊞₀_        : Hole → ⊢ Nat → Frame Nat Nat
  _⊞₁_        : ⊢ Nat → Hole → Frame Nat Nat
  test       : Hole → Frame Bool Nat

```

The Frame data type is indexed by the resulting type of the term, and the type of the subterm pending evaluation (where the hole is) respectively. Note that the frame for addition has two variants: one for when the left operand is being evaluated (thus replaced by a hole), and one for when the right operand is being evaluated, with the left value already known.

For completeness, we require a stack of frames to represent evaluation within nested subterms. This is represented as the following datatype, where the type parameter **Answer** represents the type of the final result of the computation. The remaining index is

```

data Stack (Answer : Type) : Type → Set where
  []      : Stack Answer Answer
  _∷_     : Stack Answer T1 → Frame T1 T2 → Stack Answer T2

```

The state of the abstract machine is then a pair of a stack and a mode, where it is in the process of evaluating an expression (↑) or returning a value (↓).

```

data State (Answer : Type) : Set where
  ⊢_↓_ : Stack Answer T → ⊢ T → State Answer
  ⊢_↑_ : Stack Answer T → ⊢ T → State Answer

```

The machine transitions between states and mimics how an interpreter would evaluate subexpressions in order.

```

data  $\rightarrow$  : State Answer  $\rightarrow$  State Answer  $\rightarrow$  Set where
step-` :  $\vdash$  stack  $\uparrow$  ` v  $\rightarrow$   $\vdash$  stack  $\downarrow$  v
step- $\boxplus_1$  :  $\vdash$  stack  $\uparrow$  (e1  $\boxplus$  e2)  $\rightarrow$   $\vdash$  stack :: (◊  $\boxplus_0$  e2)  $\uparrow$  e1
step- $\boxplus_2$  :  $\vdash$  stack :: (◊  $\boxplus_0$  e2)  $\downarrow$  v1  $\rightarrow$   $\vdash$  stack :: (v1  $\boxplus_1$  ◊)  $\uparrow$  e2
step- $\boxplus_3$  :  $\vdash$  stack :: (v1  $\boxplus_1$  ◊)  $\downarrow$  v2  $\rightarrow$   $\vdash$  stack  $\downarrow$  (v1 + v2)
step-test :  $\vdash$  stack  $\uparrow$  (test e)  $\rightarrow$   $\vdash$  stack :: test ◊  $\uparrow$  e
step-test1 :  $\vdash$  stack :: test ◊  $\downarrow$  n  $\rightarrow$   $\vdash$  stack  $\downarrow$  test' n
step-if :  $\vdash$  stack  $\uparrow$  (if e then e1 else e2)  $\rightarrow$ 
           $\vdash$  (stack :: (e1 else e2))  $\uparrow$  e
step-then :  $\vdash$  (stack :: (e1 else e2))  $\downarrow$  true  $\rightarrow$   $\vdash$  stack  $\uparrow$  e1
step-else :  $\vdash$  (stack :: (e1 else e2))  $\downarrow$  false  $\rightarrow$   $\vdash$  stack  $\uparrow$  e2

```

Evaluating a value simply transitions to a return state with the same value. Evaluating an addition expression pushes a frame for the left operand, and transitions to a state evaluating the left operand, with a frame containing the right operand pushed onto the stack. When the machine is in a state where the left operand has been evaluated, the frame is updated to record that the left operand has been evaluated, and the machine transitions to evaluating the right operand. When the right operand returns a value, the machine pops the frame and transitions to returning the sum of the two operands. The unary operator **test** follows the same pattern with a dedicated frame: **step-test** pushes **test** ◊ and evaluates the argument, then **step-test₁** computes **test'** n when the argument returns.

The proof for completeness is by induction on the big-step derivation; each constructor corresponds to one or more machine transitions. Because the machine pushes frames when entering subexpressions, the induction hypothesis must generalise over an arbitrary stack rather than just the empty stack.

```

complete :  $\forall$  (e :  $\vdash$  T) (v :  $\models$  T) (stack : Stack Answer T)
   $\rightarrow$  e  $\Downarrow$  v
   $\rightarrow$  ( $\vdash$  stack  $\uparrow$  e)  $\rightarrow^*$  ( $\vdash$  stack  $\downarrow$  v)

```

Here is an example of one of the cases.

```

complete (if e then e1 else e2) v stack (if-then p p1) =
  step step-if
    ( $\rightarrow^*$ transitive (complete e _ (stack :: (e1 else e2)) p)
      (step step-then
        (complete e1 v stack p1)))
complete (if e then e1 else e2) v stack (if-else p p2) =
  step step-if
    ( $\rightarrow^*$ transitive (complete e _ (stack :: (e1 else e2)) p)
      (step step-else
        (complete e2 v stack p2)))

```

In a single step, the machine begins by evaluating an expression of the form **if** e **then** e₁ **else** e₂, then transitions to a state evaluating e and pushes a stack frame containing the two branches e₁ **else** e₂. Inductively, it should be the case that the machine reduces to a return state with a boolean value v

and evidence p for $e \Downarrow v$. The machine performs a step (either **step-then** or **step-else**) to select the appropriate branch, and transitions to evaluation mode for that branch. Then inductively, we show that evaluating the selected branch reduces to a return state with the final value and evidence for that branch's evaluation.

The machine is also *correct*: if the machine reaches the return state $([], v)$ from the initial eval state $([], e)$, then $e \Downarrow v$. The proof is structurally identical to the proof for the equivalence between big-step and small-step semantics, but using the completeness and confluence of the transition relation instead of the big-step to small-step direction.

4 Exceptions

The language of arithmetic expressions is extended with exceptions. Unlike the base language where every expression terminates with a value, we now introduce the possibility of exceptional values.

4.1 Syntax

We extend the expression language with exception handling constructs:

```
data Exception : Set where
  err : Exception

data ⊢_ where
  ...
  throw      : Exception → ⊢ T
  try_catch_ : ⊢ T → (Exception → ⊢ T) → ⊢ T
```

For the purpose of this thesis, we consider a simple exception type with a single constructor `err`. However, it is possible to extend this to a more complex exception type carrying a tag and payload. A tagged exception type would allow the handler to inspect the tag and re-raise if the tag does not match, simply by having the handler return `throw ex` on an unhandled tag.

4.2 Big-Step Semantics

The big-step semantics is modified to account for both successful evaluation and exception propagation. We introduce a disjoint union: terms evaluate to either a value or an exception.

```
data Result (T : Type) : Set where
  return : ⊢ T → Result T
  raise   : Exception → Result T
```

Our relation now relates expressions to **Results**, rather than just values. For convenience we introduce aliases for the two outcomes:

- $e \Downarrow v := e \Downarrow^r \text{return } v$
- $e \Uparrow ex := e \Downarrow^r \text{raise } ex$

The pattern throughout the relation is uniform: every construct in the language that evaluates a subterm has one rule for the case where the subterm

returns a value, and a separate rule for the case where it propagates an exception. For example, in the case of addition, if either operand raises an exception, then the whole expression raises that exception; otherwise, it returns the sum of the two operands.

```

data  $\Downarrow^x$  :  $\vdash T \rightarrow \text{Result } T \rightarrow \text{Set where}$ 
   $\_ \Downarrow^x \_ : (v : \models T) \rightarrow (\backslash v) \Downarrow^x \text{return } v$ 

   $\_ \boxplus \_ : e_1 \Downarrow n_1 \rightarrow e_2 \Downarrow n_2$ 
     $\rightarrow (e_1 \boxplus e_2) \Downarrow (n_1 + n_2)$ 
   $\boxplus\text{-exn}_1 : e_1 \Uparrow \text{ex} \rightarrow (e_1 \boxplus e_2) \Uparrow \text{ex}$ 
   $\boxplus\text{-exn}_2 : e_1 \Downarrow n_1 \rightarrow e_2 \Uparrow \text{ex} \rightarrow (e_1 \boxplus e_2) \Uparrow \text{ex}$ 

   $\text{test} : e \Downarrow n \rightarrow (\text{test } e) \Downarrow \text{test}' n$ 
   $\text{test-raise} : e \Uparrow \text{ex} \rightarrow (\text{test } e) \Uparrow \text{ex}$ 

   $\text{if-raise} : e \Uparrow \text{ex} \rightarrow (\text{if } e \text{ then } e_1 \text{ else } e_2) \Uparrow \text{ex}$ 

   $\text{throw-rule} : \text{throw } \text{ex} \Downarrow^x \text{raise } \text{ex}$ 

   $\text{try-return} : e \Downarrow v \rightarrow (\text{try } e \text{ catch } h) \Downarrow v$ 
   $\text{try-catch} : e \Uparrow \text{ex} \rightarrow h \text{ ex } \Downarrow^x r \rightarrow (\text{try } e \text{ catch } h) \Downarrow^x r$ 

```

The **try-return** rule simply returns the value of the body when no exception is raised. The **try-catch** rule applies when the body raises an exception: it passes the exception to the handler h and continues with the result of evaluating the handler. The result of the handler is itself a **Result**, so the handler may either succeed or raise a further exception; which would then propagate up the expression tree.

The proof for totality is extended with the following cases.

```

...
total (throw ex) = raise ex , throw-rule
total (try e catch h) with total e
... | return v , p = return v , try-return p
... | raise ex , p = let (r , q) = total (h ex) in r , try-catch p q

```

When an exception is raised, the result produced is the exception, with the evidence for that result being the **throw-rule** constructor. When evaluating a **try-catch**, we first evaluate the body e , if it returns a value, then simply return the value, with the evidence as **try-return** rule; if it raises an exception, then we evaluate the handler h with that exception as an argument, and the result of evaluating the handler is the result of evaluating the entire expression.

4.3 Small-Step Semantics

Similarly, the small-step semantics are extended to include reduction rules to handle exception propagation. Note that all the newly introduced rules are β -rules. Once an exception is produced in some subexpression, there is no further reduction that can be performed on the rest.

```

data  $\rightarrow$  :  $\vdash T \rightarrow \vdash T \rightarrow \text{Set}$  where
...

 $\beta$ - $\boxplus$ -exn1 : (throw ex  $\boxplus$  e2)  $\rightarrow$  throw ex
 $\beta$ - $\boxplus$ -exn2 : ( $\backslash$  n1  $\boxplus$  throw ex)  $\rightarrow$  throw ex

 $\beta$ -test-exn : (test (throw ex))  $\rightarrow$  throw ex
 $\beta$ -if-exn   : (if throw ex then e1 else e2)  $\rightarrow$  throw ex

 $\xi$ -try      : e  $\rightarrow$  e'  $\rightarrow$  (try e catch h)  $\rightarrow$  (try e' catch h)
 $\beta$ -try-ok   : (try ( $\backslash$  v) catch h)  $\rightarrow$   $\backslash$  v
 $\beta$ -try-throw : (try (throw ex) catch h)  $\rightarrow$  (h ex)

```

To prove the equivalence between this and the big step semantics. First we have to lift a result back into the expression language. For values, this is just the use of the quote constructor. For exceptions, it is the throw constructor.

```

resultExpr :  $\forall$  {T : Type}  $\rightarrow$  Result T  $\rightarrow$   $\vdash T$ 
resultExpr (return v) =  $\backslash$  v
resultExpr (raise ex) = throw ex

```

In either case, it is impossible to reduce an expression resulting from the use of this function further, since there are no reduction rules that apply.

```

 $\neg$ step- $\backslash$  :  $\forall$  {T : Type} {v :  $\models T$ } {e' :  $\vdash T$ }  $\rightarrow$  ( $\backslash$  v)  $\rightarrow$  e'  $\rightarrow$   $\perp$ 
 $\neg$ step- $\backslash$  ()

 $\neg$ step-throw :  $\forall$  {T : Type} {ex : Exception} {e' :  $\vdash T$ }  $\rightarrow$  throw ex  $\rightarrow$  e'  $\rightarrow$   $\perp$ 
 $\neg$ step-throw ()

 $\neg$ step-stuck :  $\forall$  {T : Type} {r : Result T} {e' :  $\vdash T$ }  $\rightarrow$  resultExpr r  $\rightarrow$  e'  $\rightarrow$   $\perp$ 
 $\neg$ step-stuck {r = return _} =  $\neg$ step- $\backslash$ 
 $\neg$ step-stuck {r = raise _} =  $\neg$ step-throw

```

Now, the simulation proof is by induction on the big-step derivation, with a case for each constructor.

```

simulate :  $\forall$  {T : Type} {e :  $\vdash T$ } {r : Result T}  $\rightarrow$  e  $\Downarrow^r$  r  $\rightarrow$  e  $\rightarrow^*$  resultExpr r
...

simulate throw-rule           = base
simulate (try-return p)       =
   $\rightarrow^*$ transitive ( $\rightarrow^*$ try (simulate p))
  (step  $\beta$ -try-ok base)
simulate (try-catch p q)      =
   $\rightarrow^*$ transitive ( $\rightarrow^*$ try (simulate p))
  (step  $\beta$ -try-throw (simulate q))

```

For a **throw** expression, the result is an exception, so no further reduction step is necessary. For a **try-catch** expression, the result depends on the evaluation of the body and the handler. After evaluating the expression e , we can apply the ξ -rule to lift the reduction sequence to the whole **try**-expression, and then apply the appropriate β -rule depending on whether the body returned a value or raised an exception.

The lifted version of the ξ -rule for `try` simply applies the single step ξ -rule to the first reduction step, and then recursively does the same for the remaining steps.

```

→*try : ∀ {e e' : ⊢ T} {h : Exception → ⊢ T}
        → e →*e' → (try e catch h) →*(try e' catch h)
→*try base = base
→*try (step r p) = step (ξ-try r) (→*try p)

```

The proof in the other direction is identical to the proof for the base language. The only change necessary was to add cases to show that the relation is still deterministic.

4.4 Denotational Semantics

The denotational interpreter maps each expression directly to a `Result` rather than a value. For most constructs, this only requires wrapping the result in `return` if evaluating all subterms is successful, and propagating the exception otherwise.

```

interpret : ∀ {T : Type} → (⊢ T) → Result T
interpret (⊢ x) = return x
interpret (throw x) = raise x

interpret (t₁ ⊞ t₂) with interpret t₁ | interpret t₂
... | raise ex      | _ = raise ex
... | return _      | raise ex = raise ex
... | return n₁     | return n₂ = return (n₁ + n₂)

interpret (try t catch h) with interpret t
... | raise ex = interpret (h ex)
... | return v = return v

```

For `try-catch`, if the body raises an exception the interpreter immediately evaluates the handler; if the body returns a value the handler is never invoked.

The proof of equivalence between this interpreter and the big-step semantics is also identical to the base language, with the only notable difference being the wrapping of values into the result type, and of course the handling of additional constructors introduced by the exception propagation and handling.

```

simulate : ∀ {T : Type} {e : ⊢ T} {r : Result T} → e ⇝r r → interpret e ≡ r
...

simulate (throw-rule {ex}) = refl
simulate (⊞-exn₁ t₁)       rewrite simulate t₁ = refl
simulate (⊞-exn₂ t₁ t₂)    rewrite simulate t₁ | simulate t₂ = refl
simulate (try-return x)     rewrite simulate x = refl
simulate (try-catch x d)    rewrite simulate x = simulate d

```

```

simulateR : ∀ {T : Type} (e : ⊢ T) {r : Result T} → interpret e ≡ r → e ⇓r r
...
simulateR (throw x) refl = throw-rule
simulateR (try t catch h) p with interpret t in eq | p
... | raise ex | p' = try-catch (simulateR t eq) (simulateR (h ex) p')
... | return v | refl = try-return (simulateR t eq)

```

4.5 Abstract Machine

The abstract machine extends the base machine to handle exceptions. One new frame type is added: a **catch** frame, which stores the handler h and is pushed onto the stack when the machine enters a **try**-expression.

```

data Frame : Type → Type → Set where
...
catch    : (Exception → ⊢ T) → Frame T T

```

The machine now operates in three modes:

```

data State (Answer : Type) : Set where
  ⊢_↑_ : Stack Answer T → ⊢ T      → State Answer
  ⊢_↓_ : Stack Answer T → ⊢ T      → State Answer
  ⊢_<_ : Stack Answer T → Exception → State Answer

```

The eval mode ($\uparrow$) and return mode ($\downarrow$) behave as before. The *unwind mode* ($<$) carries the exception that was raised and scans the stack for the nearest **catch** frame. As for the transition rules, the machine transitions to unwind mode when a **throw** expression is evaluated.

```

data _→_ : State Answer → State Answer → Set where
...
step-throw : ⊢ stack ↑ throw ex → ⊢ stack < ex
step-try   : ⊢ stk ↑ (try e catch h)
              → ⊢ stk :: catch h ↑ e

step-catch-ok : ⊢ stk :: catch h ↓ v
                → ⊢ stk ↓ v

step-catch-exn : ⊢ stack :: catch h < ex      → ⊢ stack ↑ h ex

```

When evaluating a **try**-expression, the machine pushes a **catch** frame containing the handler h and transitions to evaluating the body e . If the machine then reaches a return state with that **catch** frame on top of the stack, it simply pops the frame and returns the value. If the machine is in the unwind mode, it evaluates the handler h .

And a rule for dropping stack frames when unwinding:

```

step-discard : ⊢ stack :: f ↑ throw ex → ⊢ stack < ex

```

However, note that such a rule is too general. In our big-step semantics, we have specific constructors that are applied when some subterm results in an exception. The following proofs would not be possible using this general

rule, as we would not be able to determine which constructor applies without inspecting the frame. Therefore, each of the constructs must have an explicit drop rule.

```

step-drop-else :  $\vdash \text{stack} :: (e_1 \text{ else } e_2) < \text{ex} \rightarrow \vdash \text{stack} < \text{ex}$ 
step-drop- $\boxplus_0$  :  $\vdash \text{stack} :: (\circ \boxplus_0 e_2) < \text{ex} \rightarrow \vdash \text{stack} < \text{ex}$ 
step-drop- $\boxplus_1$  :  $\vdash \text{stack} :: (v_1 \boxplus_1 \circ) < \text{ex} \rightarrow \vdash \text{stack} < \text{ex}$ 
step-drop-test :  $\vdash \text{stack} :: \text{test } \circ < \text{ex} \rightarrow \vdash \text{stack} < \text{ex}$ 

```

Machine completeness is split into two mutually recursive lemmas matching the two outcomes of big-step evaluation. If the stack is exhausted without finding a handler the machine is stuck, corresponding to an uncaught exception reaching the top level.

```

complete      :  $e \Downarrow v \rightarrow (\vdash \text{stk} \uparrow e) \rightarrow^* (\vdash \text{stk} \downarrow v)$ 
complete-exn  :  $e \Uparrow \text{ex} \rightarrow (\vdash \text{stk} \uparrow e) \rightarrow^* (\vdash \text{stk} < \text{ex})$ 

```

The proofs are mutually recursive due to the **try-catch** case of **complete**, which needs **complete-exn** to handle the body, and **complete-exn** must call **complete** to prove the handler case. As before, the proof is by induction on the big-step derivation, notably now including the cases for the try expression. The two proofs are almost identical, with the notable exception being the handling of the base case. Values leave the machine in a return state, while exceptions leave the machine in an unwind state.

```

complete (' _) v stk (' .v) =
  step step-' base

complete (try e catch h) v stk (try-return p) =
  step step-try
  ( $\rightarrow^*$ -transitive (complete e v (stk :: catch h) p)
   (step step-catch-ok base))

complete (try e catch h) v stk (try-catch {ex = ex} pe ph) =
  step step-try
  ( $\rightarrow^*$ -transitive (complete-exn e ex (stk :: catch h) pe)
   (step step-catch-exn
    (complete (h ex) v stk ph)))

...
complete-exn (throw _) ex stk throw-rule =
  step step-throw base

complete-exn (try e catch h) ex stk (try-catch pe ph) =
  step step-try
  ( $\rightarrow^*$ -transitive (complete-exn e _ (stk :: catch h) pe)
   (step step-catch-exn
    (complete-exn (h _) ex stk ph)))

```

Machine correctness requires a small extension of the confluence argument used in the Arithmetic chapter. There are now two possible final states either return ($\downarrow$) or unwind ($\circ$).

By totality, e evaluates big-step to some result r .

If $r = \text{return } v'$, completeness produces a canonical run to $\vdash [] \downarrow v'$; confluence with the given run yields $v = v'$.

If $r = \text{raise } ex$, completeness-exn produces a run to $\vdash [] \text{ ex}$; confluence with the given $\downarrow$ -run yields an equality between the two different constructors, which is absurd. The symmetric argument handles **correct-exn**:

correct : $\forall (e : \vdash T) (v : \models T) \rightarrow (\vdash [] \uparrow e) \rightarrow^* (\vdash [] \downarrow v) \rightarrow e \Downarrow v$
correct-exn : $\forall (e : \vdash T) (ex : \text{Exception}) \rightarrow (\vdash [] \uparrow e) \rightarrow^* (\vdash [] < ex) \rightarrow e \Uparrow ex$

```
correct e v tr with total e
... | return v' , p =
  let tr' :  $\vdash [] \uparrow e \rightarrow^* \vdash [] \downarrow v'$ 
      tr' = complete e v' [] p

      eq :  $(\vdash [] \downarrow v') \equiv (\vdash [] \downarrow v)$ 
      eq = confluence tr' tr stuck- $\downarrow$  stuck- $\downarrow$ 
  in subst ( $\lambda u \rightarrow e \Downarrow u$ ) ( $\downarrow$ -inj eq) p
... | raise ex , p =
  let tr' :  $\vdash [] \uparrow e \rightarrow^* \vdash [] < ex$ 
      tr' = complete-exn e ex [] p

      eq :  $\vdash [] < ex \equiv \vdash [] \downarrow v$ 
      eq = confluence tr' tr stuck- $<$  stuck- $\downarrow$ 
  in ex-falso ( $\downarrow \not\equiv <$  (sym eq))
```

```
correct-exn e ex tr with total e
... | return v , p =
  let tr' :  $\vdash [] \uparrow e \rightarrow^* \vdash [] \downarrow v$ 
      tr' = complete e v [] p
      eq :  $\vdash [] \downarrow v \equiv \vdash [] < ex$ 
      eq = confluence tr' tr stuck- $\downarrow$  stuck- $<$ 
  in ex-falso ( $\downarrow \not\equiv <$  eq)
... | raise ex' , p =
  let tr' :  $\vdash [] \uparrow e \rightarrow^* \vdash [] < ex'$ 
      tr' = complete-exn e ex' [] p

      eq :  $\vdash [] < ex' \equiv \vdash [] < ex$ 
      eq = confluence tr' tr stuck- $<$  stuck- $<$ 
  in subst (e  $\Uparrow$ ) ( $<$ -inj eq) p
```


5 The Simply-Typed Lambda Calculus

The Simply-Typed Lambda Calculus (STLC) extends the arithmetic expression language with λ -abstractions and function application.

5.1 Syntax and Types

STLC has two base types, together with a function type $A \Rightarrow B$.

```
data _⊢_ (Γ : Context) : Type → Set where
  Nat      : ℕ → Γ ⊢ Nat
  Bool     : ℬ → Γ ⊢ Bool
  _⊞_      : Γ ⊢ Nat → Γ ⊢ Nat → Γ ⊢ Nat
  if_then_else_ : Γ ⊢ Bool → Γ ⊢ T → Γ ⊢ T → Γ ⊢ T
  Var      : T ∈ Γ → Γ ⊢ T
  λ_       : (Γ , T1) ⊢ T2 → Γ ⊢ (T1 ⇒ T2)
  _·_      : Γ ⊢ (T1 ⇒ T2) → Γ ⊢ T1 → Γ ⊢ T2
```

Variables are represented using De Bruijn indices. A variable of type T in context Γ is a proof that $T \in \Gamma$, built from constructors $Z : T \in (\Gamma, T)$ and $S : T \in \Gamma \rightarrow T \in (\Gamma, U)$. Note that in the definition of the De Bruijn indices, the index is never the empty scope. Scope correctness is therefore guaranteed, if a value of $x : T \in \Gamma$ exists.

```
data _∈_ : Type → Context → Set where
  Z : T ∈ (Γ , T)
  S : T ∈ Γ → T ∈ (Γ , U)
```

A context is simply a list of types, and an environment δ for a context Γ assigns a value to each variable in Γ .

5.2 Big-Step Semantics

Because the language contains λ -abstractions, values include *closures*—a pairing of the environment δ^c with the body of an abstraction e^c .

```
record Closure (A B : Type) : Set where
  inductive
  constructor <_,_>
  field
    {Γc} : Context
    δc   : Environment Γc
    ec   : (Γc , A) ⊢ B
```

The **inductive** keyword is necessary because **Closure** $A\ B$ and $\models__$ are mutually recursive: $\models (A \Rightarrow B) = \text{Closure } A\ B$, while **Environment** Γ stores values of type $\models T$.

The evaluation relation is defined as a data type indexed by the environment δ , the term $e : \Gamma \vdash T$, and the value $v : \models T$. Each evaluation rule is represented as a constructor of this data type.

```
data  $\_ \vdash \_ \Downarrow \_$  : Environment  $\Gamma \rightarrow \Gamma \vdash T \rightarrow \models T \rightarrow \text{Set where}$ 
  ...
  VAR   :  $\delta \vdash \text{Var } x \Downarrow \text{lookup}_e \delta\ x$ 
  ABS   :  $\delta \vdash \lambda e \Downarrow \langle \delta, e \rangle$ 
  app   :  $\delta \vdash e_0 \Downarrow \langle \delta^c, e^c \rangle$ 
           $\rightarrow \delta \vdash e_1 \Downarrow v_1$ 
           $\rightarrow (\delta^c, v_1) \vdash e^c \Downarrow v$ 
           $\rightarrow \delta \vdash e_0 \cdot e_1 \Downarrow v$ 
```

Variables are evaluated by looking up their value in the environment. Evaluating a λ -abstraction produces a closure containing the captured environment and the body of the abstraction. The application case evaluates the first operand to a closure, evaluates the second operand to a value, and then evaluates the body of the closure in an extended environment where the captured environment is extended with the argument value.

The STLC is known to be strongly normalising: every well-typed term evaluates to a normal form, that is a term that cannot be further reduced. As is the case for the other languages discussed in this thesis, normal forms are values.

```
{-# TERMINATING #-}
total :  $\forall (e : \Gamma \vdash T) \rightarrow \exists (\lambda v \rightarrow \delta \vdash e \Downarrow v)$ 
  ...
  total { $\delta = \delta$ } (Var  $x$ ) = lookupe  $\delta\ x$  ‘ VAR  $x$ 
  total ( $\lambda e$ ) =  $\langle \_, e \rangle$  ‘ ABS  $e$ 
  total ( $e_1 \cdot e_2$ ) with total  $e_1$  | total  $e_2$ 
  ... | ( $\langle \delta^c, e^c \rangle$  ‘  $p$ ) | ( $v_2$  ‘  $q$ ) with total { $\delta = \delta^c, v_2$ }  $e^c$ 
  ... | ( $v$  ‘  $r$ ) =  $v$  ‘ app  $p\ q\ r$ 
```

As it is not obvious to the termination checker that the body of the closure is structurally smaller. Evaluating a λ -abstraction returns a closure containing the captured environment and the body of the abstraction. Much like the definition in the big-step semantics, the case for variables simply looks up the value of the variable in the environment. The definition is marked with **{-# TERMINATING #-}** stemming from the call to **total** on the body of the closure e^c in the application case.

5.3 Small-Step Semantics

We now attempt to address the small-step semantics for STLC. Recall that the small step relation is defined on syntactic terms, and not "values" as in the big-step semantics. A term is *normal* if it cannot take a further step; the **Normal** predicate characterises this class:

```

data Normal :  $\Gamma \vdash T \rightarrow \text{Set}$  where
  nat      : Normal (Nat n)
  bool     : Normal (Bool b)
  closure  : Normal ( $\lambda e$ )

data  $\rightarrow_{\rightarrow}$  :  $\Gamma \vdash T \rightarrow \Gamma \vdash T \rightarrow \text{Set}$  where
  ...
   $\beta$ -app      : ( $\lambda e \cdot v$ )  $\rightarrow$  e [ v ]
   $\xi$ -app-left :  $e_1 \rightarrow e_1' \rightarrow (e_1 \cdot e_2) \rightarrow (e_1' \cdot e_2)$ 
   $\xi$ -app-right : Normal v  $\rightarrow e_2 \rightarrow e_2' \rightarrow (v \cdot e_2) \rightarrow (v \cdot e_2')$ 

```

Note that unlike the big-step semantics, the small-step semantics performs substitution in the case of β -reduction of function application, that is to say: the body of the closure is evaluated, but the bound variable is replaced with the argument value, rather than being looked up in an environment.

Unfortunately, this causes the proof to be much more difficult, as we would have to prove that looking up the variable in the environment is equivalent to this substitution. The following section discusses an incomplete attempt made to prove the equivalence.

The first is addressing the statement of the theorem itself. If $\delta \vdash e \Downarrow v$, then it must be possible to reinterpret v back into a term.

```

embed :  $\forall \{T : \text{Type}\} \rightarrow \models T \rightarrow \emptyset \vdash T$ 
embed {Nat} n = Nat n
embed {Bool} b = Bool b
embed {T = T1  $\Rightarrow$  T2}  $\langle \delta^c, e^c \rangle = \lambda ?$ 

```

It is apparent closures should be embedded into an expression of the form (λe^c) , but the issue is that of the captured environment δ^c . The body of the closure e^c is defined in the context Γ^c , but the embedding requires that the body of the λ -abstraction is defined in the context $(\emptyset, T_1)$.

Still, assuming this definition can be completed, the relation is written out as:

```

simulate :  $\forall \{\Gamma\} \{\delta : \text{Environment } \Gamma\} \{e : \emptyset \vdash T\} \{v : \models T\}$ 
   $\rightarrow \emptyset \vdash e \Downarrow v$ 
   $\rightarrow e \rightarrow^* \text{embed } v$ 

simulateR :  $\forall \{e : \emptyset \vdash T\} \{v : \models T\} \rightarrow e \rightarrow^* \text{embed } v \rightarrow \emptyset \vdash e \Downarrow v$ 

```

Both functions cannot be completed without filling in the definition of `embed`.

An alternate formulation of the STLC with the "quote" constructor for values was also part of the attempt. The source code is provided in the "STLC-Alt" directory accompanying this thesis. The two theorems then can be formulated in terms of the quote:

```

simulate :  $\rightarrow \emptyset \vdash e \Downarrow v \rightarrow e \rightarrow^* \text{quote } v$ 
simulateR :  $\forall \{e : \emptyset \vdash T\} \{v : \models T\} \rightarrow e \rightarrow^* \text{quote } v \rightarrow \emptyset \vdash e \Downarrow v$ 

```

That however, does not solve the issue of proving that substitution is equivalent to the looking semantics.

$$\begin{aligned} \delta \vdash s \ [\ \backslash \ v \] \Downarrow v_2 \\ \rightarrow \delta \vdash s \ [\ \backslash \ \langle \delta^c, e^c \rangle \cdot \backslash \ v_1 \] \Downarrow v_2 \end{aligned}$$

There are ways of aligning the two semantics, by adopting a different representation of variables or so. However, none of this is pursued in this thesis.

5.4 Denotational Semantics

Two denotational interpretations of STLC are presented: a direct semantics utilizing Agda's function type and an explicit closure semantics.

Direct Semantics

The direct semantics maps each type T to the domain of Agda types $\llbracket T \rrbracket$, most notably, interpreting function types as Agda function types: $\langle\langle A \Rightarrow B \rangle\rangle = \langle\langle A \rangle\rangle \rightarrow \langle\langle B \rangle\rangle$.

```

⟦_⟧ : Type → Set
⟦ Bool  ⟧ = B
⟦ Nat   ⟧ = N
⟦ A ⇒ B ⟧ = ⟦ A ⟧ → ⟦ B ⟧

interpret : Env Γ → Γ ⊢ T → ⟦ T ⟧
...
interpret δ (Var x)           = lookupd δ x
interpret δ (λ x e)           = λ z → interpret (δ , z) e
interpret δ (e1 · e2)       = (interpret δ e1) (interpret δ e2)

```

Unsurprisingly, variables are interpreted by looking up their value in the environment; λ -abstractions in our languages are interpreted as Agda functions that take an argument $z : \langle\langle A \rangle\rangle$ and interpret the body of the abstraction in the extended environment; function application is then simply interpreted as function application.

The environment must be redefined to account for this change in the value domain, so that it now maps variables to values in the direct semantics. Environments are still defined inductively on the Context, as a chain of products of values.

```

Env : Context → Set
Env ∅ = T
Env (Γ , T) = Env Γ × ⟦ T ⟧

lookupd : ∀ {Γ T} → Env Γ → T ∈ Γ → ⟦ T ⟧
lookupd (Γ , x) Z = x
lookupd (Γ , x) (S n) = lookupd Γ n

```

Closure Semantics

Rather than interpreting function types into functions of our meta-language, the closure semantics represents them as explicit closures (as defined above,

see 5.2). The **evaluate** and **apply** functions are defined by mutual recursion: **evaluate** recurses on the structure of the term and delegates function application to **apply**; **apply** unpacks a closure $\langle \delta^c, e^c \rangle$, extends the saved environment with the argument value, and calls **evaluate** on the body.

```
{-# TERMINATING #-}
apply      : Closure T1 T2 → ⊢ T1 → ⊢ T2
evaluate : Environment Γ → Γ ⊢ T → ⊢ T

...
evaluate δ (Var x)           = lookupe δ x
evaluate δ (λ e)             = ⟨ δ , e ⟩
evaluate δ (e1 · e2)       = apply (evaluate δ e1) (evaluate δ e2)

apply ⟨ δc , ec ⟩ v = evaluate (δc , v) ec
```

Note the use of the terminating pragma, which much like the totality proof for the big-step semantics, is necessary to convince the termination checker that the recursive call in **apply** is on a structurally smaller term.

Logical Relations

The domain of values is different in the case of function types. So the following does not type check:

```
evaluate δ (λ e) ≡ interpret δ (λ e)
```

Specifically, the environment on the left side is of type **Environment** Γ , which the domain with explicit closures, while the environment on the right side is of type **Env** Γ , the domain with direct semantics.

Instead, the two denotational interpretations are related by a logical relations argument. The relation $v \sim_{\langle T \rangle} u$ states that a closure-semantics value $v : \vdash T$ and a direct-semantics value $u : \ll T \gg$ are semantically equivalent at type T :

```
relates : (A : Type) → ⊢ A → ⟨⟨ A ⟩⟩ → Set
x1 ~⟨ Nat ⟩~ x2 = x1 ≡ x2
x1 ~⟨ Bool ⟩~ x2 = x1 ≡ x2
f1 ~⟨ T ⇒ U ⟩~ f2 = ∀ {a1 a2}
  → a1 ~⟨ T ⟩~ a2
  → apply f1 a1 ~⟨ U ⟩~ f2 a2
```

For base types the relation is simply propositional equality. For function types, two functions are related if applying them to related arguments produces related results. Note the use of the **apply** function to produce a term from an application of an explicit closure f_1 in the definition of the relation for function types.

The central theorem states that the two denotational semantics agree on every term under related environments:

$$\delta \sim_{\langle \Gamma \rangle} \delta' \implies \text{evaluate } \delta e \sim_{\langle T \rangle} \text{interpret } \delta' e$$

5.5 Abstract Machine

The abstract machine for STLC extends the arithmetic machine with frames for function application. The key distinction from the machines introduced in the previous chapters is that the environment is carried *with the machine state*.

The three application frames are:

```
data Frame : Type → Type → Set where
  app1 : Hole → Γ ⊢ A      → Frame B (A ⇒ B)
  app2 : ⊢ (A ⇒ B) → Hole   → Frame B A
  app3 : Environment Γ → ⊢ (A ⇒ B) → ⊢ A → Hole → Frame B B
```

Function application $e_1 \cdot e_2$ proceeds through three successive frames:

1. **app₁**: push a frame recording the unevaluated argument e_2 , and evaluate e_1 to obtain the closure.
2. **app₂**: push a frame recording the obtained closure, and evaluate e_2 to obtain the argument value v_2 .
3. **app₃**: switch the active environment to the closure's saved environment δ^c , push a frame recording the caller's environment δ , and evaluate the closure body e^c . When the evaluation of the body e^c finishes, **app₃** restores the caller's environment.

The corresponding transition rules are:

```
data →_ : State Answer → State Answer → Set where
  step-var : (δ ⊢ stack ↑ Var x)
    → (δ ⊢ stack ↓ lookupe δ x)
  step-λ    : (δ ⊢ stack ↑ (λ e))
    → (δ ⊢ stack ↓ ⟨ δ , e ⟩)
  step-·    : (δ ⊢ stack ↑ e1 · e2)
    → (δ ⊢ stack :: app1 ∘ e2 ↑ e1)
  step-·1  : (δ ⊢ stack :: app1 ∘ e2 ↓ v1)
    → (δ ⊢ stack :: app2 v1 ∘ ↑ e2)
  step-·2  : (δ ⊢ stack :: app2 ⟨ δc , ec ⟩ ∘ ↓ v2)
    → ((δc , v2) ⊢ stack :: app3 δ ⟨ δc , ec ⟩ v2 ∘ ↑ ec)
  step-·3  : (δ ⊢ stack :: app3 δ' f v2 ∘ ↓ v3)
    → (δ' ⊢ stack ↓ v3)
```

Variable lookup (**step-var**) reads the value directly from the environment. Evaluating a λ -abstraction (**step-λ**) captures the current environment δ together with the body e into a closure value $\langle \delta, e \rangle$ which is returned. Evaluating a function application proceeds by first evaluating the left operand into a closure, popping the stack frame and pushing a new one containing the closure, and evaluating the argument. Once the argument is evaluated, the machine switches to the closure's saved environment, inserting the value of the argument and evaluates the body of the closure. When the body of the function

is evaluated, the machine restores the call-site environment and returns the result.

All that remains is the proof of equivalence between the abstract machine and the big-step semantics. For completeness, the proof is by induction on big-step derivation.

```

complete : (δ ⊢ stack ↑ e →* δ ⊢ stack ↓ v)
complete δ stack (Var _) v (VAR _) = step step-var base
complete δ stack (λ _ ) v (ABS _) = step step-λ base
complete δ stack (e0 · e1) v (app {δc = δc} {ec} {v1} p p1 p2) =
  proof
    δ ⊢ stack ↑ (e0 · e1)
  →⟨ step-· ⟩
    δ ⊢ stack :: app1 ∘ e1 ↑ e0
  →⟨ complete δ (stack :: app1 ∘ e1) e0 ⟨ δc , ec ⟩ p ⟩
    δ ⊢ stack :: app1 ∘ e1 ↓ ⟨ δc , ec ⟩
  →⟨ step-·1 ⟩
    δ ⊢ stack :: app2 ⟨ δc , ec ⟩ ∘ ↑ e1
  →⟨ complete δ (stack :: app2 ⟨ δc , ec ⟩ ∘) e1 v1 p1 ⟩
    δ ⊢ stack :: app2 ⟨ δc , ec ⟩ ∘ ↓ v1
  →⟨ step-·2 ⟩
    (δc , v1) ⊢ stack :: app3 δ ⟨ δc , ec ⟩ v1 ∘ ↑ ec
  →⟨ complete (δc , v1) (stack :: app3 δ ⟨ δc , ec ⟩ v1 ∘) ec v p2 ⟩
    (δc , v1) ⊢ stack :: app3 δ ⟨ δc , ec ⟩ v1 ∘ ↓ v
  →⟨ step-·3 {v1 = ⟨ δc , ec ⟩} ⟩
    δ ⊢ stack ↓ v
  ■

```

For the case of variables, values and λ terms, applies the single corresponding step and returns the required value. The case for application proceeds as it was previously described above.

The proof for correctness is again, identical to the one for the arithmetic machine, with the same confluence argument. We use the totality of the big-step semantics to obtain a value v' such that $\delta \vdash e \Downarrow v'$. Using the completeness lemma, we obtain a run from $\delta \vdash \Box \uparrow e$ to $\delta \vdash \Box \downarrow v'$.

Both v and v' are results (values), therefore the runs $\delta \vdash \Box \uparrow e \rightarrow^* \delta \vdash \Box \downarrow v$ and $\delta \vdash \Box \uparrow e \rightarrow^* \delta \vdash \Box \downarrow v'$ are final states.

Confluence with the given run from $\delta \vdash \Box \uparrow e$ to $\delta \vdash \Box \downarrow v$ yields $v = v'$, which completes the proof.

6 Related Work

Hutton [Hut23] presents a systematic route from semantics to an abstract machine for a version of the language of arithmetic expressions. The method is different from the one used in this thesis. Rather than starting from the structure of big-step derivations and reifying suspended evaluation contexts as a stack of frames, Hutton begins with an evaluator, rewrites it into continuation-passing style, and defunctionalizes the resulting continuation-passing version into a first-order datatype. Despite these differences, there is a correspondence between the two developments: a frame in the present thesis is equivalent to a defunctionalized continuation in Hutton’s account. Correctness is proven by construction.

Hutton and Wright [HW06] extend the approach above with exceptions. The source language is similar to the one studied in here: arithmetic expressions with untagged, payload-free exceptions and handlers. The basic continuation-based setup described in the previous paper is used, with modifications: the type of the continuation is modified to account for the possibility that evaluation raises an exception rather than returning a value. They [HW04] have also shown a derivation of a verified compiler from the language to a stack-based abstract machine. Of particular relevance is the treatment of handlers: the code for a compiled handler is pushed onto a stack of handlers, its “extent” is delimited by mark and unmark instructions. An exception causes the machine to unwind until a matching handler is found. The analogous mechanism used in this thesis is the unwind mode in the exceptional abstract machine, and more generally to the explicit propagation rules in the source semantics that make exceptional control flow visible. Hutton et al. [PH21] present a formalization of the aforementioned compiler in Agda, without explicit proofs of correctness.

7 Conclusion

This thesis presented a process for the systematic derivation of an interpreter in the form of an abstract machine. In particular, for arithmetic expressions, arithmetic expressions with exceptions, and the simply typed lambda calculus.

The derivation proceeds in a systematic manner:

- representing the evaluation of terms explicitly, via frames that record what has already been evaluated, together with holes representing terms pending evaluation and unevaluated terms;
- defining abstract-machine states as a stack of frames operating in three modes: eval, return, and, in the case of exceptions, unwind;
- defining transition rules between machine states by mimicking the in-order evaluation of the term; and

Finally, each derived machine is proven to be semantically consistent with the big-step semantics. The first direction (completeness) is inductively proven by showing that the execution trace of the abstract machine follows that of a big-step evaluation derivation. The second direction (correctness) is proven by the confluence argument, using the totality of the big-step semantics of our language to obtain a run of the machine to a value v' , and then showing that any other run producing v must converge to the same result.

We also showed that the semantics are compositional; we can derive an equivalence between other forms of semantics by considering the big-step semantics as an intermediary.

Several limitations of the current approach point to directions for future work. First, this approach assumes that all programs are total: both the big-step semantics and the totality proof used in the correctness argument inherently do not account for non-terminating programs. Addressing this would require either moving to a coinductive big-step semantics or pivoting to small-step semantics as the form of semantics from which others are derived. Second, the totality proof for the STLC currently relies on a termination pragma, meaning it is accepted by Agda on trust rather than verified structurally; finding a way to eliminate this pragma remains open. The equivalence proof between the small-step and big-step semantics for the STLC remains open as well.

Bibliography

- [Bru75] Nicolaas Govert de Bruijn. “Lambda Calculus Notation with Nameless Dummies, a Tool for Automatic Formula Manipulation, with Application to the Church-Rosser Theorem”. In: *Journal of Symbolic Logic* 40 (1975), pp. 470–470. URL: <https://api.semanticscholar.org/CorpusID:117106648>.
- [Hut23] Graham Hutton. “Programming language semantics: It’s easy as 1,2,3”. In: *Journal of Functional Programming* 33 (2023). URL: <https://api.semanticscholar.org/CorpusID:247581669>.
- [HW04] Graham Hutton and Joel Wright. “Compiling Exceptions Correctly”. In: *Mathematics of Program Construction*. Ed. by Dexter Kozen. Berlin, Heidelberg: Springer Berlin Heidelberg, 2004, pp. 211–227. ISBN: 978-3-540-27764-4.
- [HW06] Graham Hutton and Joel Wright. “Calculating an Exceptional Machine”. In: *Trends in Functional Programming volume 5*. Ed. by Hans-Wolfgang Loidl. Selected papers from the Fifth Symposium on Trends in Functional Programming, Munich, November 2004. Intellect, Feb. 2006.
- [PH21] Mitchell Pickard and Graham Hutton. “Calculating Dependently-Typed Compilers”. In: *Proceedings of the ACM on Programming Languages* 5.ICFP (Aug. 2021).
- [Pie02] Benjamin C. Pierce. *Types and Programming Languages*. 1st. The MIT Press, 2002. ISBN: 0262162091.
- [Ros22] “WebAssembly Core Specification”. Version 2.0. In: (Apr. 19, 2022). Ed. by Andreas Rossberg. URL: <https://www.w3.org/TR/wasm-core-2/>.
- [Stu16] Aaron Stump. “Verified Functional Programming in Agda”. In: *ACM Books*. 2016. URL: <https://api.semanticscholar.org/CorpusID:61393966>.
- [Wad15] Philip Wadler. “Propositions as types”. In: *Commun. ACM* 58.12 (Nov. 2015), pp. 75–84. ISSN: 0001-0782. DOI: [10.1145/2699407](https://doi.org/10.1145/2699407). URL: <https://doi.org/10.1145/2699407>.